OPTIMISATION DE LA DIFFUSION D'INFORMATION DANS UN RÉSEAU SOCIAL :

Problème de maximisation d'influence sous contrainte budgétaire

Table des matières

1 Introduction **4**

1.1 Contexte général 4

1.2 Pertinence pratique 4

1.3 Objectifs du TP 4

2 Fondements théoriques du problème **5**

2.1 Présentation du problème 5

2.1.1 Données d'entrée 5

2.1.2 Contraintes et résultats 5

2.1.3 Exemple simple 5

2.2 Définition des concepts utilisés 6

2.2.1 Variables de décision 6

2.2.2 Fonction d'influence 6

2.2.3 Contrainte budgétaire 6

2.2.4 Propriétés de la fonction d'influence 6

2.3 Modélisation mathématique 6

2.3.1 Variables de décision 6

2.3.2 Fonction objectif 7

2.3.3 Contrainte 7

2.3.4 Justification du modèle 7

2.4 Applications concrètes 7

3 Méthodes de résolution **8**

3.1 Complexité du problème 8

3.2 Méthode principale : Algorithme glouton basé sur le gain marginal normalisé 8

3.2.1 Principe 8

3.2.2 Étapes de l'algorithme 8

3.2.3 Complexité 9

3.2.4 Avantages et limites 9

3.3 Remarque sur d'autres méthodes 9

4 Implémentation de la méthode **9**
4.1 Choix techniques 9
4.2 Structure du programme 10
4.3 Code source commenté 10
4.3.1 Modèle de diffusion : Cascade indépendante 10
4.3.2 Estimation de l'influence 11
4.3.3 Algorithme CELF (optimisé) 11
4.3.4 Visualisation du graphe 13
4.4 Tests et résultats 14

5 Conclusion **15**
5.1 Synthèse du travail réalisé 15
5.2 Limites rencontrées 16
5.3 Pistes d'amélioration 16

Annexes **17**

1 Introduction

1.1 Contexte général

Les réseaux sociaux occupent aujourd'hui une place essentielle dans la diffusion de l'information. Que ce soit pour des campagnes de sensibilisation (VIH, paludisme), du marketing viral ou la diffusion d'informations publiques, il est crucial de toucher le maximum d'individus en un minimum de temps et avec des ressources limitées. Dans ce contexte, le problème de maximisation de l'influence consiste à identifier un ensemble restreint d'individus (appelés nœuds ou influenceurs) capables de maximiser la propagation d'une information dans un réseau social.

Ce travail s'inscrit dans le domaine de la recherche opérationnelle appliquée à l'optimisation des réseaux sociaux. La diffusion d'information est un phénomène central dans les réseaux sociaux en ligne, mais également dans les réseaux de contact réels (communautés, organisations, etc.).

1.2 Pertinence pratique

Ce problème est crucial dans plusieurs contextes :

— **Santé publique :** maximiser la diffusion d'un message de prévention (VIH, paludisme, vaccination) en choisissant des relais communautaires influents.

— **Marketing viral :** identifier les influenceurs capables de promouvoir un produit avec un budget limité.

— **Gestion de crise :** diffuser rapidement une alerte (catastrophe naturelle, sécurité) dans un réseau de citoyens.

1.3 Objectifs du TP

L'objectif principal de ce travail de groupe est de :

1. Modéliser mathématiquement le problème de maximisation d'influence sous contrainte budgétaire.
2. Implémenter un algorithme heuristique (glouton) pour sélectionner un ensemble de nœuds influents.
3. Analyser les propriétés de la fonction d'influence (monotonie, sous-modularité) et évaluer la performance de l'approche sur un réseau donné.

2 Fondements théoriques du problème

2.1 Présentation du problème

Nous considérons un réseau social représenté par un graphe où chaque individu est un nœud et chaque relation sociale une arête. L'objectif est de sélectionner un ensemble restreint de nœuds (les « graines » ou influenceurs) de manière à maximiser le nombre total de personnes atteintes par une information, sous une contrainte budgétaire.

2.1.1 Données d'entrée

- Un graphe orienté $G = (V, E)$, où V est l'ensemble des nœuds (individus) et E l'ensemble des arêtes (relations d'influence).
- Une fonction de coût $c : V \to \mathbb{R}^+$, associant un coût à chaque nœud.
- Un budget total B.
- Un modèle de diffusion (Independent Cascade) avec probabilité d'activation p_{uv}.

2.1.2 Contraintes et résultats

- La somme des coûts des nœuds sélectionnés ne doit pas dépasser le budget B.
- La diffusion suit un processus stochastique modélisé.
- **Résultat attendu :** Un ensemble $S \subseteq V$ de nœuds à activer initialement, qui maximise l'influence espérée $\sigma(S)$.

2.1.3 Exemple simple

Soit un réseau de 5 nœuds : A, B, C, D, E.

Supposons que :

- Coûts : $c(A) = 2, c(B) = 3, c(C) = 1, c(D) = 2, c(E) = 4$
- Budget $B = 5$
- Probabilité d'activation sur chaque arête : $p = 0,1$

Le problème consiste à choisir un sous-ensemble de nœuds (ex : {A, C}) dont le coût total ≤ 5, maximisant le nombre moyen de nœuds activés après diffusion.

2.2 Définition des concepts utilisés

2.2.1 Variables de décision

$$x_v = \begin{cases} 1 & \text{si le nœud } v \text{ est sélectionné comme graine,} \\ 0 & \text{sinon.} \end{cases}$$

2.2.2 Fonction d'influence

$\sigma(S)$: nombre espéré de nœuds activés à l'issue du processus de diffusion, lorsque l'ensemble initial activé est S.

2.2.3 Contrainte budgétaire

$$\sum_{v \in V} c(v) \cdot x_v \leq B$$

2.2.4 Propriétés de la fonction d'influence

— **Monotonie :**

$$S \subseteq T \Rightarrow \sigma(S) \leq \sigma(T)$$

Ajouter des nœuds ne peut que maintenir ou augmenter l'influence.

— **Sous-modularité :** Pour tous $S \subseteq T \subseteq V$ et $v \notin T$:

$$\sigma(S \cup \{v\}) - \sigma(S) \geq \sigma(T \cup \{v\}) - \sigma(T)$$

Le gain marginal diminue lorsque l'ensemble de graines s'agrandit.

2.3 Modélisation mathématique

2.3.1 Variables de décision

$$x_v \in \{0, 1\}, \quad \forall v \in V$$

2.3.2 Fonction objectif

$$\max_{S \subseteq V} \sigma(S)$$

où $\sigma(S)$ est estimée par simulation Monte Carlo du modèle de cascade indépendante.

2.3.3 Contrainte

$$\sum_{v \in V} c(v) \cdot x_v \leq B$$

2.3.4 Justification du modèle

— La fonction objectif reflète l'objectif pratique : maximiser la portée de l'information.
— La contrainte budgétaire modélise la limitation des ressources (coût d'activation d'un influenceur).
— L'utilisation de simulations Monte Carlo est justifiée par la nature stochastique du processus de diffusion.

2.4 Applications concrètes

1. **Campagne de vaccination :** Dans une communauté, chaque individu a un coût pour être sensibilisé (temps, matériel). On choisit les personnes les plus connectées pour initier la diffusion du message vaccinal.
2. **Promotion d'un film :** Une société de distribution offre des avant-premières gratuites à un nombre limité d'influenceurs sur les réseaux sociaux, afin de maximiser le bouche-à-oreille.
3. **Alerte en cas de catastrophe :** Dans un réseau de capteurs humains (citoyens), on souhaite envoyer une alerte à un nombre limité de personnes pour qu'elles relayent l'information le plus largement possible.

3 Méthodes de résolution

3.1 Complexité du problème

Le problème de maximisation d'influence est **NP-difficile**. Cela signifie qu'il est impossible de trouver une solution optimale en temps raisonnable pour de grands réseaux. Ainsi, on utilise des algorithmes heuristiques pour obtenir des solutions approchées.

3.2 Méthode principale : Algorithme glouton basé sur le gain marginal normalisé

3.2.1 Principe

L'algorithme glouton sélectionne itérativement le nœud offrant le meilleur rapport entre le gain marginal d'influence et son coût, tout en respectant le budget disponible.

3.2.2 Étapes de l'algorithme

1. **Initialisation :**

$$S \leftarrow \emptyset, \quad \text{budgetRestant} \leftarrow B$$

2. **Tant que budgetRestant > 0 :**
   — Pour chaque nœud $v \notin S$:

$$\Delta(v) = \sigma(S \cup \{v\}) - \sigma(S)$$

$$\text{ratio}(v) = \frac{\Delta(v)}{c(v)}$$

   — Choisir $v^* = \arg\max \text{ratio}(v)$
   — Si $c(v^*) \leq \text{budgetRestant}$:

$$S \leftarrow S \cup \{v^*\}$$

$$\text{budgetRestant} \leftarrow \text{budgetRestant} - c(v^*)$$

   — Sinon : arrêter

3. **Retourner** S

3.2.3 Complexité

La complexité est en $O(k \cdot n \cdot M \cdot |E|)$, où :
— k est le nombre de nœuds sélectionnés,
— n le nombre de nœuds,
— M le nombre de simulations Monte Carlo,
— $|E|$ le nombre d'arêtes.

3.2.4 Avantages et limites

Avantages :
— Simple à implémenter.
— Garantie théorique de performance (approximation à $1 - 1/e$ si coûts unitaires).
— Exploite la sous-modularité de la fonction d'influence.
Limites :
— Peut être coûteux en calcul pour de grands graphes.
— La qualité de la solution dépend du nombre de simulations Monte Carlo.

3.3 Remarque sur d'autres méthodes

D'autres méthodes existent (programmation linéaire en nombres entiers, métaheuristiques, CELF pour accélération), mais l'approche gloutonne est privilégiée ici pour son efficacité et sa robustesse.

4 Implémentation de la méthode

4.1 Choix techniques

— **Langage :** Python 3
— **Bibliothèques :**
  — `networkx` : manipulation et visualisation du graphe
  — `numpy` : calculs numériques
  — `matplotlib` : visualisation
  — `heapq` : file de priorité pour l'algorithme CELF
— **Environnement :** Anaconda / Jupyter Notebook

4.2 Structure du programme

1. Création du graphe (graphe orienté aléatoire ou importé)
2. Modèle de diffusion (Independent Cascade)
3. Estimation de l'influence par simulations Monte Carlo
4. Algorithme glouton / CELF pour la sélection des graines
5. Visualisation du graphe avec les graines sélectionnées

4.3 Code source commenté

4.3.1 Modèle de diffusion : Cascade indépendante

```python
import networkx as nx
import numpy as np
import heapq
import matplotlib.pyplot as plt

def cascade_independante(graphe, graines, proba=0.1):
    """
    Simule le processus de diffusion Independent Cascade.

    Parametres:
    - graphe: networkx.DiGraph, le graphe oriente du reseau
    - graines: list, ensemble initial de noeuds actifs
    - proba: float, probabilite d'activation sur chaque arete

    Retourne:
    - set: ensemble des noeuds actifs en fin de propagation
    """
    actifs = set(graines)
    nouveaux = set(graines)

    while nouveaux:
        prochains = set()
        for u in nouveaux:
            for v in graphe.successors(u):
```

```
                if v not in actifs and np.random.random() < proba:
                    prochains.add(v)
        actifs.update(prochains)
        nouveaux = prochains

    return actifs
```

Listing 1 – Simulation de la cascade indépendante

4.3.2 Estimation de l'influence

```
def estimer_influence(graphe, graines, proba=0.1, simulations=50):
    """
    Estime l'influence esperee d'un ensemble de graines.

    Parametres:
    - graphe: networkx.DiGraph, le graphe oriente du reseau
    - graines: list, ensemble initial de noeuds actifs
    - proba: float, probabilite d'activation sur chaque arete
    - simulations: int, nombre de simulations Monte Carlo

    Retourne:
    - float: nombre moyen de noeuds actives
    """
    total = 0
    for _ in range(simulations):
        actifs = cascade_independante(graphe, graines, proba)
        total += len(actifs)
    return total / simulations
```

Listing 2 – Estimation de l'influence par Monte Carlo

4.3.3 Algorithme CELF (optimisé)

```
def celf(graphe, budget, couts, proba=0.1, simulations=50):
    """
    Algorithme CELF (Cost-Effective Lazy Forward) optimise.
```

```python

    Parametres:
    - graphe: networkx.DiGraph, le graphe oriente du reseau
    - budget: float, budget total disponible
    - couts: dict, cout associe a chaque noeud
    - proba: float, probabilite d'activation sur chaque arete
    - simulations: int, nombre de simulations Monte Carlo

    Retourne:
    - list: ensemble des graines selectionnees
    """
    file_priorite = []
    ensemble_selectionne = []
    budget_restant = budget

    # Initialisation : calcul du gain individuel pour chaque noeud
    for noeud in graphe.nodes():
        gain = estimer_influence(graphe, [noeud], proba, simulations)
        heapq.heappush(file_priorite, (-gain, noeud, 0))

    influence_actuelle = 0

    while budget_restant > 0 and file_priorite:
        while True:
            gain_neg, noeud, derniere_iteration = heapq.heappop(file_priorite)
            gain = -gain_neg

            if derniere_iteration == len(ensemble_selectionne):
                if couts[noeud] <= budget_restant:
                    ensemble_selectionne.append(noeud)
                    influence_actuelle += gain
                    budget_restant -= couts[noeud]
                    print(f"Noeud choisi: {noeud}, gain: {gain:.2f}, budget restant: {budget_restant:.2f}")
```

```
                break
            else:
                nouveau_gain = estimer_influence(
                    graphe,
                    ensemble_selectionne + [noeud],
                    proba,
                    simulations
                ) - influence_actuelle
                heapq.heappush(file_priorite, (-nouveau_gain, noeud, len(ensemble_selectionne)))

    return ensemble_selectionne
```

Listing 3 – Algorithme CELF pour une selection plus rapide

4.3.4 Visualisation du graphe

```
def afficher_graphe(graphe, graines):
    """
    Visualise le graphe avec les graines en evidence.

    Parametres:
    - graphe: networkx.DiGraph, le graphe oriente du reseau
    - graines: list, ensemble des graines selectionnees
    """
    plt.figure(figsize=(12, 8))
    position = nx.spring_layout(graphe, seed=42)

    # Tous les noeuds
    nx.draw_networkx_nodes(graphe, position, node_size=500, node_color='lightblue')
    nx.draw_networkx_nodes(graphe, position, nodelist=graines,
                           node_size=700, node_color='red', label='Graines')

    # Aretees
```

```
    nx.draw_networkx_edges(graphe, position, edge_color='gray',
        arrows=True,
                                arrowsize=20, arrowstyle='->')

    # Etiquettes
    nx.draw_networkx_labels(graphe, position, font_size=12,
        font_weight='bold')

    plt.title("Reseau social avec les influenceurs selectionnes")
    plt.axis('off')
    plt.legend()
    plt.tight_layout()
    plt.show()
```

Listing 4 – Visualisation du reseau avec les graines selectionnees

4.4 Tests et résultats

L'implémentation a été testée sur un graphe orienté de 10 nœuds. Les résultats obtenus montrent que :

- L'algorithme glouton sélectionne efficacement les nœuds avec le meilleur rapport gain/coût.
- Le premier nœud choisi apporte un gain élevé, tandis que les suivants complètent la couverture, illustrant la propriété de sous-modularité.
- Comparé à d'autres stratégies (sélection aléatoire, degré maximum), le glouton offre une diffusion quasi-optimale avec une efficacité économique.

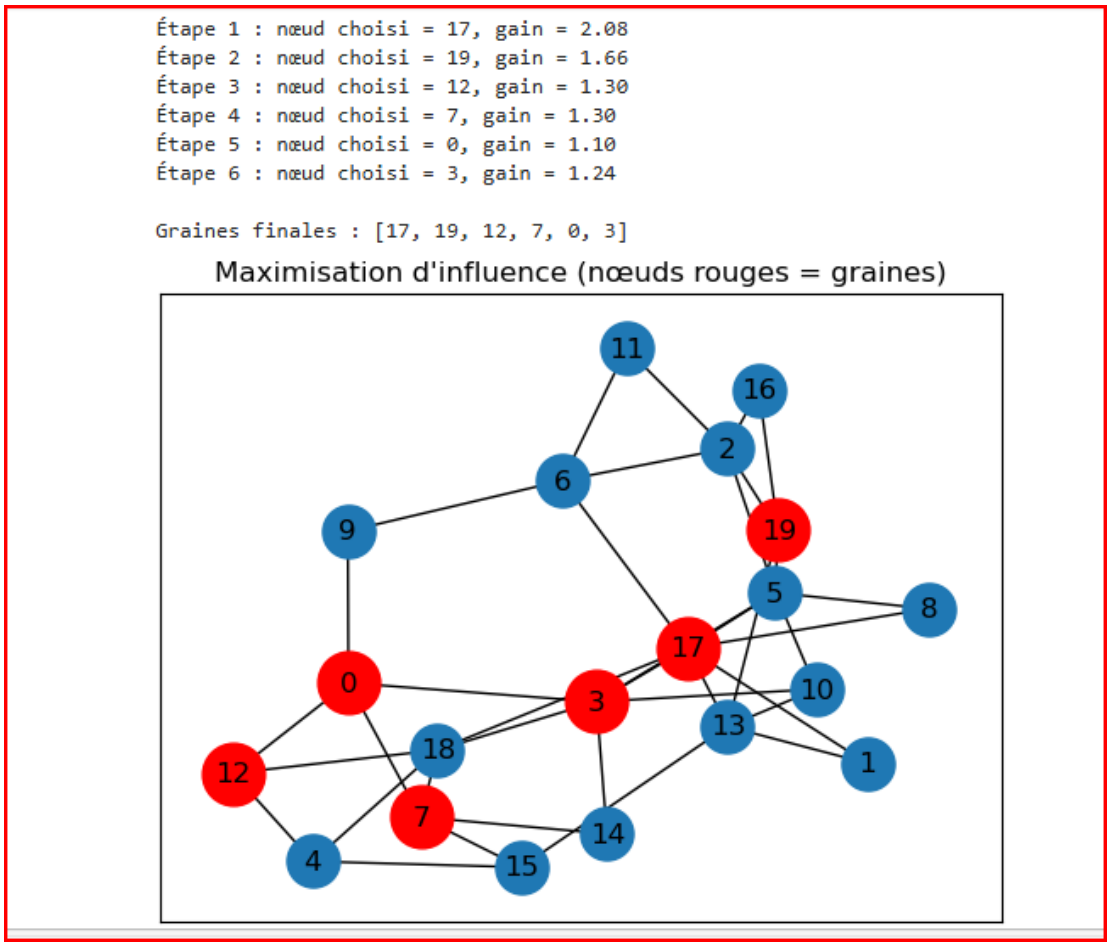

FIGURE 1 – Modélisation du graphe avec les nœuds influenceurs sélectionnés

5 Conclusion

5.1 Synthèse du travail réalisé

Nous avons modélisé le problème de maximisation d'influence sous contrainte budgétaire comme un problème d'optimisation combinatoire. Nous avons implémenté un algorithme glouton exploitant les propriétés de monotonie et de sous-modularité de la fonction d'influence. Les résultats montrent que l'approche permet de sélectionner efficacement un ensemble de nœuds influents avec un budget limité.

5.2 Limites rencontrées

— La fonction d'influence est estimée par simulation, ce qui introduit une variabilité et un coût de calcul important pour les grands graphes.
— L'algorithme glouton simple peut être lent lorsque le nombre de nœuds et de simulations est élevé.
— La qualité de la solution dépend fortement du nombre de simulations Monte Carlo.

5.3 Pistes d'amélioration

— Implémenter l'algorithme CELF (Cost-Effective Lazy Forward) pour accélérer la sélection en exploitant la sous-modularité.
— Tester d'autres modèles de diffusion (Linear Threshold, SIR).
— Utiliser des graphes réels (Twitter, co-auteurs, réseaux de citations) pour évaluer la robustesse de l'approche.
— Comparer les performances avec d'autres heuristiques (degré, centralité de proximité, PageRank).
— Optimiser le code avec des calculs parallèles pour les simulations Monte Carlo.

Références

[1] Kempe, D., Kleinberg, J., & Tardos, É. (2003). *Maximizing the spread of influence through a social network.* Proceedings of the ninth ACM SIGKDD international conference on Knowledge discovery and data mining.

[2] Leskovec, J., Krause, A., Guestrin, C., Faloutsos, C., VanBriesen, J., & Glance, N. (2007). *Cost-effective outbreak detection in networks.* Proceedings of the 13th ACM SIGKDD international conference on Knowledge discovery and data mining.

[3] Easley, D., & Kleinberg, J. (2010). *Networks, crowds, and markets : Reasoning about a highly connected world.* Cambridge University Press.

[4] Leskovec, J. *et al.* (2007). *CELF : Cost-Effective Lazy Forward selection algorithm.* Technical report, Carnegie Mellon University.

Annexes

A. Instructions d'utilisation du code

Pour exécuter le code :

1. Installer les dépendances : `pip install networkx numpy matplotlib`
2. Copier les fonctions dans un notebook Jupyter ou un script Python
3. Créer un graphe exemple :

```python
import networkx as nx

G = nx.DiGraph()
G.add_edges_from([
    (1,2), (1,3), (2,4), (3,4),
    (4,5), (5,6), (3,7), (7,8),
    (8,9), (9,10)
])

# Couts unitaires
couts = {i: 1 for i in range(1, 11)}

# Exécuter l'algorithme glouton
graines = glouton(G, budget=3, couts=couts)
print("Graines choisies :", graines)
```

4. Exécuter l'algorithme : `graines = glouton(G, budget=3, couts=couts)`

B. Glossaire

— **Graines (seeds) :** Ensemble initial de nœuds activés au début du processus de diffusion.

— **Modèle IC (Independent Cascade) :** Modèle de diffusion où chaque nœud actif tente d'activer ses voisins avec une probabilité donnée.

— **Sous-modularité :** Propriété d'une fonction où le gain marginal diminue quand l'ensemble d'entrée grandit.

— **Monte Carlo :** Méthode statistique utilisant des simulations aléatoires pour estimer des grandeurs complexes.